written is not so important. Once a test is written, the path to the next one is usually more clear: find some way to specify something the code doesn't do yet.

When TDD Needs Some Help

Test-driven development is helpful, but it won't solve all of your development problems by itself. There are areas where developer testing doesn't apply or doesn't work very well.

I mentioned one case already: developer tests are not very good at determining whether the application is behaving correctly according to requirements. Strict TDD is not great at acceptance testing. There are, however, automated tools that do try to tackle acceptance testing. Within the Rails community, the most prominent of these are Capybara and Cucumber; see Chapter 8, *Integration Testing with Capybara and Cucumber*, on page 151. Acceptance testing can be integrated with TDD—you'll sometimes see this called *outside-in testing*. That's a perfectly valid and useful test paradigm, but it's an extension of the classic TDD process.

Testing your application assumes that you know the right answer to specify. And although you sometimes have clear requirements or a definitive source of correct output, other times you don't know what exactly the program needs to do. In this exploratory mode, TDD is less beneficial, because it's hard to write tests if you don't know what assertions to make about the program. Often this lack of direction happens during initial development or during a proof of concept. I also find myself in this position a lot when view-testing—I don't know what to test for until I get some of the view up and visible.

The TDD process has a name for the kind of exploratory code you write while trying to figure out the needed functionality: *spike*, as in, "I don't know if we can do what we need with the Twitter API; let's spend a day working on a spike for it." When working in spike mode, TDD is generally not used, but code written during the spike is not expected to be used in production; it's just a proof of concept to be thrown away and replaced with a version written using TDD.

When view-testing, or in other nonspike situations where I'm not quite sure what output to test for, I often go into a "test-next" mode, where I write the code first but in a TDD-sized small chunk, and then immediately write the test. This works as long as I make the switch between test and code frequently enough to get the benefit of having the code and the test inform each other's design.

TDD is not a complete solution for verifying your application. I've already talked about acceptance tests; it's also true that TDD tends to be thin in terms of the quantity of unit tests written. For one thing, in a strict TDD